

Pulse Programming Language

Comparison & Evaluation Study

Pulse vs. Python -- Syntax, AI/ML Support, Performance, and Suitability

Final Year Project

Daud Anjum

2026-09-13

1. Introduction

This study compares Pulse against Python, the general-purpose language Pulse's syntax is most directly modeled on across four dimensions: language syntax and features, AI/ML support and library requirements, execution-time performance, and overall suitability for Pulse's stated educational and AI/ML objectives. Results are reported as measured; where Pulse performs worse than Python, that is stated plainly rather than reframed, consistent with the objective, non-promotional aim of this report.

Python was chosen as the comparison baseline because Pulse's syntax is explicitly modeled on it, and because Python is the dominant language for AI/ML development today (via numpy, pandas, and scikit-learn), making it the most meaningful reference point for evaluating whether Pulse's AI-native design achieves a genuine advantage.

2. Methodology

2.1 Syntax and Feature Comparison

Equivalent language constructs were written in both languages and compared directly (Section 3).

2.2 Execution-Time Benchmarking

Five equivalent-workload program pairs were authored, one Pulse (.pul) and one or two Python (.py) equivalents per benchmark, using identical input sizes and identical algorithms. Each program was executed as a separate OS process and timed externally via Python's `time.perf_counter()` wrapping a subprocess call, not self-reported by either language's own internal timing across 5 trials per program, with the median trial reported to reduce the influence of OS scheduling noise. Benchmarks covered: a tight while-loop, recursive function calls, list comprehension, and matrix multiplication (compared against both numpy directly and a naive pure-Python implementation, since Pulse's tensor type is itself numpy-backed).

2.3 Objectives Evaluation

Each objective from the project's original proposal document (Section 1.3, "Objectives of the Proposed Language") was assessed against concrete evidence produced during development and testing: working sample programs, automated test results, and the benchmark data in this report.

3. Syntax and Feature Comparison

3.1 Core Language Constructs

Variable assignment and printing

```
x = 10
print(f'value: {x}')
```

Identical in both languages.

Function definition

```
def add(a, b):
    return a + b
```

Identical syntax in both languages, including default arguments and `*args` varargs.

Classes

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f'{self.name} speaks')
```

Near-identical; Pulse uses a static keyword before def instead of Python's @staticmethod decorator.

Tuples and multiple return values

A key divergence identified during this project: Python supports tuple literals and comma-based multiple return values; Pulse does not.

Python	Pulse (workaround required)
pair = ("a", 1)	pair = ["a", 1] # list, not tuple
return a, b	return [a, b] # then index into result

Tensor / matrix literals

Python (requires numpy)	Pulse (built-in)
import numpy as np m = np.array([[1, 2], [3, 4]])	m = @[[1, 2], [3, 4]]
c = a @ b	c = a @ b

Pulse's tensor literal and @ matmul operator are built into the grammar; Python requires an external library import for equivalent functionality.

3.2 Feature Comparison Summary

Feature	Python	Pulse
Tuples	Supported	Not supported
Multiple return values	Supported (comma syntax)	Not supported (return a list instead)
Tensor/matrix literal syntax	Not built in (needs numpy)	Built in (@[[...]])
Matrix multiply operator	Needs numpy's @ on arrays	Native @ operator
ML model training	Needs scikit-learn install	Built in (models module)
Static methods	@staticmethod decorator	static keyword
Pattern matching	match/case (3.10+)	match/case
List comprehensions	Supported	Supported
Lambdas	Supported	Supported
Exception handling	try/except/else/finally	try/except/else/finally
Calling external Python code	N/A (is Python)	Supported via pybridge module

4. AI/ML Support and Library Requirements

The central design claim of Pulse is that AI/ML functionality is available without external library installation. This was verified directly: the regression, classification, and chatbot sample programs run with only `import models` / `import preprocess` / `import metrics` / `import datasets`, all of which are part of Pulse's own stdlib with no pip install step required by the end user.

Capability	Python (typical stack)	Pulse
Numerical arrays / tensors	pip install numpy	Built in
Linear/Logistic regression	pip install scikit-learn	Built in (models module)
Train/test split, scaling	pip install scikit-learn	Built in (preprocess module)
Accuracy, R2, confusion matrix, etc.	pip install scikit-learn	Built in (metrics module)
Built-in datasets (iris, diabetes, etc.)	pip install scikit-learn	Built in (datasets module)
Calling into the other ecosystem	N/A	pybridge module (this project)

The tradeoff is ecosystem breadth: Python's AI/ML libraries (scikit-learn, PyTorch, TensorFlow, pandas, etc.) represent a vastly larger and more mature feature set, deep learning, GPU acceleration, distributed training none of which Pulse's stdlib currently replicates. Pulse's built-in coverage is limited to the models/preprocess/metrics/datasets surface demonstrated in this project's sample programs (linear/logistic regression, KMeans clustering) plus general tensor arithmetic.

5. Execution-Time Benchmarks

All timings below are the median of 5 externally-timed trials per program (see Section 2.2 for methodology). Full raw results:

Benchmark	Pulse (s)	Python (s)	Ratio (Pulse/Python)
Loop summation (200,000 iterations)	1.9001	0.0569	33.4x
Recursive fibonacci (fib(24))	2.3041	0.0325	70.8x
List comprehension (50,000 elements)	0.4150	0.0279	14.9x
Matrix multiply 80x80 x20 (numpy-backed)	0.4226	0.1854	2.3x
Matrix multiply 80x80 x20 (pure Python baseline)	0.4391	0.6769	0.6x

5.1 Interpretation

For general-purpose interpreted control flow, a tight loop and recursive function calls with no tensor/AI operations involved, Pulse is substantially slower than CPython: 33x slower for the loop benchmark and 71x slower for recursive fibonacci. This is consistent with Pulse's tree-walking AST interpreter design (each statement/expression is re-dispatched through Python-level visitor methods on every execution) versus CPython's compiled bytecode with an optimized eval loop. The list comprehension benchmark shows a smaller but still substantial 15x gap.

The picture changes for tensor operations, which are the operations most relevant to Pulse's stated AI-native purpose. Pulse's tensor matmul is only 2.3x slower than calling numpy directly in Python, the overhead here is Pulse's interpreter dispatch cost around what is ultimately the same underlying numpy call, not a reimplementaion of matrix multiplication. More notably, Pulse's tensor matmul is roughly 1.6x faster than a naive pure-Python triple-nested-loop implementation of the same operation, because Pulse delegates to numpy's vectorized implementation while the pure-Python baseline does not.

This produces a nuanced, objective conclusion: Pulse's "Fast Program Execution" objective (Section 1.3 of the proposal) is substantiated specifically for tensor/AI workloads, the class of computation the language was designed around but does not hold for general-purpose interpreted code, where Pulse is markedly slower than Python. A developer using Pulse for what it was designed for (tensor/model-centric AI code) will not experience a meaningful performance penalty versus writing the numpy equivalent by hand; a developer writing performance-sensitive general-purpose logic in Pulse (large loops, deep recursion) will.

6. Evaluation Against Original Objectives

Each objective below is drawn verbatim from the project's proposal document (Section 1.3) and assessed against evidence gathered during this project's development and testing.

AI-Native Syntax

Objective (proposal text)	Provide built-in support for tensors, matrices, and common AI operations, reducing the need for external libraries.
Status	Met
Evidence	Tensor literals (@[[...]]), the @ matmul operator, and built-in models / preprocess / metrics / datasets stdlib modules are implemented and confirmed working (linear regression, classification, and chatbot sample programs all train and predict without any external pip install beyond the interpreter's own dependencies).

Fast Program Execution

Objective (proposal text)	Enable the interpreter to execute Pulse programs efficiently, completing typical AI tasks within minimal runtime.
Status	Partially Met
Evidence	For tensor/AI-specific operations -- the actual "AI tasks" this objective targets -- Pulse is competitive: tensor matmul is only 2.3x slower than calling numpy directly (interpreter dispatch overhead on top of a numpy call), and is actually ~1.6x faster than a naive pure-Python triple-loop implementation, because Pulse delegates to numpy while pure Python does not. However, for general-purpose interpreted control flow (loops, recursion), Pulse is 15x-71x slower than CPython, reflecting its tree-walking (non-bytecode-compiled) interpreter design. The objective holds for the AI/tensor workloads it was aimed at, but not for general-purpose computation.

Sample AI Programs

Objective (proposal text)	Allow developers to run example AI applications, like chatbots or ML models, in minutes for demonstration purposes.
Status	Met

Evidence	A working linear regression, classification (iris), and intent-classification chatbot were each authored and run successfully end-to-end, each under 100 lines of Pulse source and each training in well under one second.
-----------------	--

Readable and Simple Syntax

Objective (proposal text)	Maintain Python-like readability with offering DSL-level precision, reducing code complexity compared to general-purpose languages.
Status	Met
Evidence	Pulse syntax is near-identical to Python for control flow, function/class definitions, and expressions (see Section 3). Notable simplifications from Python were identified during testing: no tuple literals, no multi-value return via comma syntax, and static replaces <code>@staticmethod</code> -- minor deviations that do not undermine overall readability.

Error Handling and Debugging

Objective (proposal text)	Provide syntax validation and runtime feedback, helping developers detect and fix errors within minimal time.
Status	Met
Evidence	The <code>PulseError</code> hierarchy (<code>PulseLexError</code> , <code>PulseSyntaxError</code> , <code>PulseSemanticError</code> , <code>PulseRuntimeError</code>) reports line/column, source context with a caret pointer, and optional hints. An opt-in structured JSON Lines error logging facility (<code>--log</code>) was additionally implemented during this project phase.

Backend Integration

Objective (proposal text)	Use Python for computations and AI function execution, ensuring seamless execution of ML tasks.
Status	Met
Evidence	The <code>pybridge</code> <code>stdlib</code> module (<code>pybridge.import_module</code> / <code>pybridge.load_file</code>) allows Pulse code to call installed Python packages and user-authored Python files directly, with automatic value conversion at the boundary. Confirmed working end to end, with 30 passing automated regression tests.

Rapid Prototyping

Objective (proposal text)	Accelerate the development of small AI models, reducing setup and coding time compared to standard workflows.
Status	Met
Evidence	The classification and regression sample programs required no environment setup beyond running the interpreter -- no virtual environment, no pip install of <code>scikit-learn</code> / <code>pandas</code> / <code>numpy</code> by the end user -- since these are bundled into Pulse's own <code>stdlib</code> implementation.

7. Strengths, Weaknesses, Limitations, and Possible Improvements

7.1 Strengths

- Genuine AI-native syntax: tensor literals and the `@ matmul` operator are first-class grammar, not library calls.
- Zero-install AI/ML workflow: regression, classification, and clustering sample programs run with no pip install step.
- Tensor operations are competitively fast because they delegate to numpy rather than reimplementing numerical computation in the interpreter.
- Structured, informative error messages (line/column, source context, carets, hints) support the language's debugging objective.
- Working Python interoperability (pybridge) allows falling back to the full Python ecosystem when Pulse's own stdlib is insufficient.
- Python-like syntax minimizes the learning curve for the language's intended audience.

7.2 Weaknesses and Limitations

- General-purpose interpreted code (loops, recursion) is 15x-71x slower than CPython, limiting Pulse's suitability for performance-sensitive non-tensor computation.
- No tuples and no multi-value return via comma syntax; both require workarounds (lists) that add minor friction versus idiomatic Python.
- No text/NLP stdlib support; the chatbot sample program required hand-rolled bag-of-words vectorization rather than a built-in text pipeline, despite chatbots being one of the objective's named example use cases.
- The tree-walking interpreter design (no bytecode compilation or caching) is the root cause of the general-purpose performance gap and would require a more substantial architecture change to close.
- pybridge executes arbitrary Python code with full interpreter-process privileges; suitable for an educational/demonstration context but not a production or multi-tenant security boundary.
- Pulse's built-in ML surface (linear/logistic regression, KMeans) is narrow compared to scikit-learn's breadth, and has no equivalent to PyTorch/TensorFlow-style deep learning or GPU acceleration.
- Two of the six sample AI/ML programs (clustering, tensor_ops) have not been independently re-verified against the live interpreter as of this report; see the companion Testing and Validation Report, Section 7.5.

7.3 Possible Improvements

- Bytecode compilation (or at minimum AST-node caching) to close the general-purpose performance gap versus CPython.
- Add tuple support and multi-value return syntax to reduce friction versus idiomatic Python.
- Expand the stdlib with a lightweight text/NLP module, given chatbots are an explicitly named target use case.
- Add an opt-in sandboxing mode for pybridge (e.g. an allow-list of importable modules) for contexts where arbitrary code execution is undesirable.
- Broaden the models module's coverage (e.g. decision trees, basic neural networks are referenced in the language's own --info documentation but were not exercised in this evaluation).
- Introduce automated code coverage measurement (e.g. pytest-cov) to complement the current test-count-based reporting.

8. Conclusion: Suitability for Educational and AI/ML Purposes

Pulse achieves its stated objective of AI-native syntax and zero-install AI/ML prototyping: the regression, classification, and chatbot sample programs demonstrate that a developer can go from a blank file to a trained model in well under 100 lines of code and well under one second of training time, without installing numpy, pandas, or scikit-learn. This directly supports the language's educational and rapid-prototyping goals.

The benchmark data shows this holds specifically for tensor/AI-centric code, where Pulse's overhead versus hand-written numpy is modest (2.3x) and its overhead versus naive pure-Python numerical code is negative (i.e. Pulse is faster). However, Pulse is not a general-purpose performance competitor to Python: interpreted control flow outside of tensor operations is 15x-71x slower, a direct consequence of its tree-walking interpreter design.

Taken together, Pulse is well suited to its intended purpose as an educational, AI-native DSL for prototyping and demonstrating small AI/ML programs -- the exact scenarios its objectives and sample programs target. It is not, and does not claim to be, a general-purpose replacement for Python; its performance profile and narrower stdlib make it unsuitable for performance-critical or general-purpose production workloads outside the AI/tensor domain it was designed for. This is a reasonable and expected tradeoff for a language whose explicit goal is DSL-level precision for AI tasks rather than general-purpose competitiveness, and the evidence gathered in this study supports that positioning rather than a broader claim of superiority over Python.